

Thresholding

Dr. Ei Moh Moh Aung

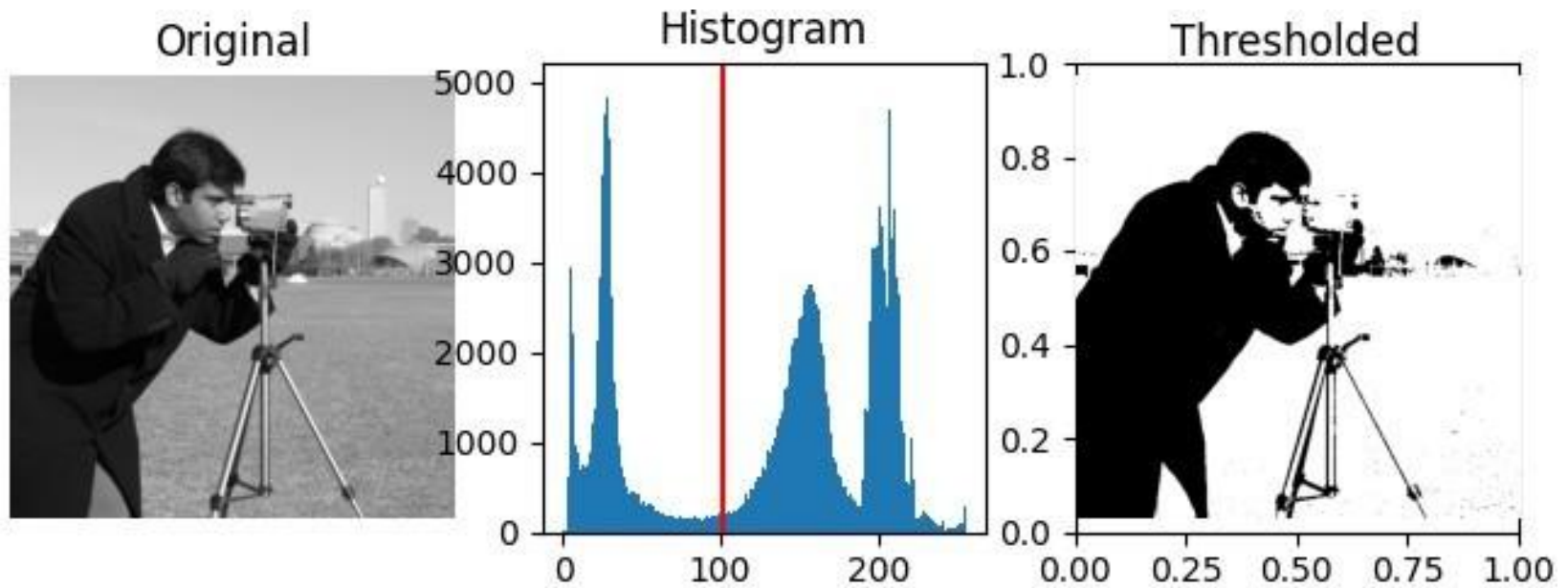
UIT



Thresholding

- Thresholding is a fundamental technique in computer vision used to **segment an image into foreground and background objects**.
- It works by **classifying each pixel** in the image as either belonging to the object of interest (foreground) or the background based on their intensity values.

Thresholding



Thresholding : Process

- 1. Image Representation:
 - Images are typically represented as matrices, where each pixel has an intensity value (e.g., grayscale: 0-255).
- 2. Threshold Selection:
 - A threshold value is chosen. **Pixels with intensity values greater than or equal to the threshold** are classified as **foreground**, while those below are classified as background.
- 3. Binary Image Creation:
 - The resulting image becomes a binary image, containing only 0s (background) and 1s (foreground).

Thresholding : Process

- Types of Thresholding:
 - **Global Thresholding**: Uses a single threshold value for the **entire image**.
 - **Local Thresholding**: Adapts the threshold based on the **local pixel neighborhood**.
 - **Adaptive Thresholding**: Employs various strategies to dynamically adjust the threshold based on local image statistics.

Thresholding : Process

- Applications:
 - **Object detection and segmentation:** Isolating objects of interest from the background.
 - **Feature extraction:** Identifying specific features within objects.
 - **Image analysis:** Measuring object properties like size, shape, and texture.
 - **Noise reduction:** Simplifying images by removing irrelevant details.

Thresholding : Process

- Advantages:
 - Simple and computationally efficient.
 - Effective for **segmenting high-contrast images** with clear foreground-background separation.
- Disadvantages:
 - Sensitive to noise and uneven illumination.
 - May not perform well for **complex images with subtle intensity variations**.

Binary Thresholding

- Binary thresholding, as you already know, is a fundamental image processing technique that **segments** an image into **foreground and background objects** based on their intensity values.
- It achieves this by classifying each pixel as either belonging to the object of interest (foreground) or the background **based on a chosen threshold value**.



Binary Thresholding

```
ret, output = cv2.threshold
    (source, threshold_value, max_value, threshold_type))
```

$$\text{output} \begin{cases} 255 & \text{if } \text{pixel} > \text{threshold} \\ 0 & \text{if } \text{pixel} \leq \text{threshold} \end{cases}$$

Threshold_type

- THRESH_BINARY
- THRESH_BINARY_INV

THRESH_BINARY

- small/dark → BLACK
- large/bright → WHITE

Binary Thresholding

- Key Concepts:
 - **Image Representation:** Images are typically represented as matrices, where each pixel has an intensity value (e.g., grayscale: 0-255).
 - **Threshold Value:** This value separates foreground (intensity $\geq$ threshold) from background pixels.
 - **Binary Image:** The resulting image contains only 0s (background) and 1s (foreground).

Binary Image



Inverse Binary Thresholding

- Inverse binary thresholding, also known as inverted thresholding, is a simple image processing technique that **inverts the results of standard binary thresholding**.
- Inverse binary thresholding takes the output of standard thresholding and simply flips the pixel values. This means:
 - Foreground pixels (**white**) become background (**black**).
 - Background pixels (**black**) become foreground (**white**).

Inverse Binary Thresholding

- Effect:
 - This inversion effectively swaps the roles of foreground and background in the image. Objects that were **originally dark appear bright**, and the **background becomes dark**.
- Applications:
 - Useful when the object of interest has lower intensity than the background (e.g., dark text on a white page).
 - Can be combined with other image processing techniques like **edge detection for feature extraction**.

Otsu Thresholding

- Otsu's thresholding is a popular and effective **automatic thresholding technique** used in image processing to separate an image into foreground and background based on pixel intensity.
 - It aims to find the threshold value that **maximizes the between-class variance**, essentially creating the best possible separation between the two classes (foreground and background).
- `thresh, output = cv2.threshold`
`( image, threshold_value, max_value, cv2.THRESH_OTSU )`

Otsu Thresholding

- **Histogram Analysis:** Otsu's method analyzes the image's histogram, which represents the distribution of pixel intensities.
- **Class Probabilities:** The histogram is divided into two sections at each possible threshold value, calculating the probabilities of pixels belonging to each class (foreground and background).
- **Within-Class Variance:** For each threshold, the within-class variance for both foreground and background classes is calculated. This measures how spread out the pixels are within each class.
- **Optimal Threshold:** The threshold that minimizes the total within-class variance is chosen as the optimal threshold. This means the classes are most distinct at this threshold value.

-

END